Aplicación móvil Ferretería Pescado Volador

Decisiones de diseño móvil, complementos utilizados y manejo de permisos · Laboratorio 11

Proyecto	Pescado_Volador_Movil
Variante de Ionic	Vue (Ionic 9, Vue 3.5, Vue Router 5, Pinia)
Capa nativa	Capacitor 8 · plataforma Android (`com.pescadovolador.movil`)
API	FerreteriaPescadoVolador (Laravel + Sanctum)
Fecha	8 de noviembre de 2026

Contenido

1. Decisiones de diseño móvil
2. Complementos utilizados
3. Manejo de permisos

Las rutas de archivo son relativas a la raíz del repositorio de la aplicación móvil.

1. Decisiones de diseño móvil

1.1 Navegación y estructura

Decisión	Justificación
Pestañas inferiores: Productos, Ventas y Recepción.	Quedan al alcance del pulgar y cada pestaña conserva su propia pila de páginas. Cada rol solo ve las pestañas que puede usar (por ejemplo, el cajero no ve Recepción).
Cuatro tipos de página: inicio de sesión, listado, detalle y formulario. Crear y editar comparten el mismo formulario.	Evita duplicar campos y validaciones; la ruta define el modo (`/nuevo` o `/:id/editar`).
Carga diferida de cada página y rutas privadas por defecto, protegidas por un guardián.	Arranque más rápido y ninguna pantalla privada queda expuesta por olvido.
Botón atrás de Android: después del login no vuelve a esa pantalla. En la pantalla raíz pregunta «¿Desea salir de la aplicación?». Con la cámara abierta, solo la cierra.	Se comporta como espera el usuario de Android y evita salir de la app o volver al login por accidente.

1.2 Interfaz

Decisión	Justificación
Solo **componentes de Ionic** (encabezados, listas, tarjetas, campos, alertas y avisos).	Aspecto nativo en Android y comportamiento ya resuelto: teclado, desplazamiento y áreas seguras.
Identidad visual del front web: misma paleta, fuente Clarity City incluida en la app y franja bajo el encabezado. **Modo oscuro** opcional que se recuerda.	El sistema se reconoce igual en web y en móvil; la fuente funciona sin internet.
Listados en **tarjetas** con **desplazamiento infinito** y **recarga al deslizar**.	Son los patrones táctiles habituales para listas largas, sin botones de paginación.
Estados visibles: carga (`ion-spinner`), vacío con una acción y error con **Reintentar**, todos como tarjetas de Ionic.	El usuario siempre sabe qué pasa y puede recuperarse sin salir de la pantalla.
Confirmación antes de las acciones que no se pueden deshacer (eliminar un producto, confirmar una recepción, salir).	En una pantalla táctil es fácil tocar por error.
Accesibilidad: zonas táctiles de al menos 44 px, etiquetas en los botones de ícono y mensajes con `role="status"` o `role="alert"`.	Uso cómodo con el dedo y compatibilidad con lectores de pantalla.

1.3 Tamaños de pantalla y áreas seguras

- **De borde a borde**: `index.html` declara `viewport-fit=cover`. Capacitor 8 entrega los márgenes del sistema y Ionic los aplica en el encabezado (barra de estado y recorte de la cámara) y en la barra de pestañas (barra de gestos). El visor de la cámara suma esos márgenes por su cuenta.

- **Diseño adaptable**: la grilla de productos usa 2 columnas en teléfono y 3 o 4 en tableta, y los formularios se centran con un ancho máximo para no estirarse en pantallas grandes.

1.4 Datos, sesión y conectividad

Decisión	Justificación
Estado en **almacenes de Pinia**; las pantallas nunca llaman a la API directamente.	Separa la interfaz de los datos y facilita las pruebas.
Un **único punto de salida** a la API (axios con interceptores) que agrega el token y, ante un 401, cierra la sesión y lleva al login.	Ninguna pantalla maneja el token a mano.
Token en almacenamiento cifrado; nada sensible en Preferences.	Preferences y el almacenamiento del navegador no están cifrados.
Primero los datos guardados: sin conexión se muestra el último listado con la fecha en que se guardó, y las escrituras se bloquean.	La app sigue siendo útil sin red y no se pierden cambios que no se pueden enviar.
Si el token vence mientras se llena una venta, **el formulario se conserva** y se recupera después del login.	No se pierde trabajo del cajero.

1.5 Uso del hardware: cámara

Decisión	Justificación
Cámara para la **recepción de compras**: el bodeguero escanea la etiqueta, la app completa el producto y su stock, elige la compra pendiente y confirma la cantidad.	Es la tarea de bodega donde escribir códigos largos (13 a 20 dígitos) causa más errores. El backend suma el stock y registra el Kardex.
Solo códigos **Code 128**.	Es el formato de las etiquetas que genera el backend; limitar el formato acelera la lectura y evita leer otros códigos por error.
Interfaz de escaneo **propia** (`startScan`), con marco, instrucción y botón Cancelar.	Mantiene la identidad de la app y permite controlar el permiso de cámara. El escáner de Google (`scan`) no usa ese permiso y depende de un módulo de Google Play.
Siempre se puede **escribir el código**.	Sirve sin cámara, con el permiso negado o con una etiqueta dañada. En el navegador es la única opción.
La cantidad recibida debe **coincidir** con la esperada; si no, se avisa y no se puede confirmar.	Aplica en la pantalla la misma regla que el backend, antes de enviar.

2. Complementos utilizados

Complementos (plugins) de Capacitor instalados, con la versión de `package.json`:

Complemento	Versión	Uso en la app	Dónde se usa
`@capacitor-mlkit/barcode-scanning`	8.2.2	Cámara y lectura de códigos Code 128 (ML Kit con el modelo incluido en la app). Consulta y pide el permiso de cámara y abre los ajustes de la app.	`src/servicios/escaner.ts` (único punto de uso)
`@aparajita/capacitor-secure-storage`	8.0.1	Guarda la sesión (token) cifrada con AES/GCM, con una llave que vive en el Android Keystore.	`src/servicios/almacenSeguro.ts`
`@capacitor/network`	8.0.1	Estado de la conexión al abrir la app y avisos de cambio, para usar la caché y bloquear escrituras.	`src/stores/red.ts`
`@capacitor/preferences`	8.0.1	Datos no sensibles: caché de los listados con su fecha y el tema elegido.	`src/servicios/cache.ts`, `src/stores/tema.ts`
`@capacitor/app`	8.1.2	Botón atrás, salir de la app, y los eventos `pause` (cierra la cámara) y `resume` (vuelve a leer el permiso al regresar de Ajustes).	`src/App.vue`, `src/composables/useEscanerCodigo.ts`
`@capacitor/core` y `@capacitor/android`	8.6.0	Puente entre la web y Android, detección de plataforma nativa, márgenes de las áreas seguras y registro del estado de los permisos.	Toda la app
`@capacitor/status-bar`, `keyboard` y `haptics`	8.0.x	Vienen con la plantilla de Ionic. La app no los usa directamente; keyboard ajusta la vista cuando aparece el teclado.	Sin uso en el código

Criterios de elección

- **Compatibles con Capacitor 8** y mantenidos: todos declaran `@capacitor/core >= 8`.
- **Escáner:** se eligió `@capacitor-mlkit/barcode-scanning` porque expone la API completa de permisos (`checkPermissions`, `requestPermissions`, `openSettings`), permite filtrar por formato y funciona sin conexión.
- **Almacenamiento seguro:** cifra con una llave del Keystore del sistema, que no se puede extraer. El plugin oficial Preferences no cifra.
- **Un archivo por complemento** (servicio o *store*): el resto de la app no depende del plugin, lo que facilita cambiarlo o simularlo en las pruebas unitarias.

3. Manejo de permisos

3.1 Permisos de Android declarados

Permiso	Tipo	Declarado en	Para qué
`INTERNET`	Normal	`android/app/src/main/AndroidManifest.xml`	Comunicarse con la API.
`ACCESS_NETWORK_STATE`	Normal	Manifiesto del plugin Network (se combina al compilar)	Saber si hay conexión.
`VIBRATE`	Normal	Manifiesto del plugin Haptics	Lo agrega el plugin de la plantilla.
`CAMERA`	**Peligroso**	`AndroidManifest.xml`	Escanear el código de barras.

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.CAMERA" />
<uses-feature android:name="android.hardware.camera" android:required="false" />
```

Los permisos **normales** los concede Android al instalar la app. `CAMERA` es **peligroso**: además de declararlo, hay que pedirlo mientras la app corre. La cámara se marca como no obligatoria (`required="false"`) para que la app se pueda instalar en equipos sin cámara, donde el código se escribe a mano.

3.2 Permiso de cámara en tiempo de ejecución

- **Cuándo se pide:** solo cuando el usuario toca «Escanear código de barras» en la pantalla Recepción, nunca al abrir la app. Así el pedido tiene contexto.
- **Cómo se decide qué hacer:** antes de abrir la cámara se consulta el estado del permiso (`checkPermissions`) y la app actúa según la tabla y la figura 1 (`src/servicios/escaner.ts` y `src/composables/useEscanerCodigo.ts`).

Estado (Capacitor)	Significado	Qué hace la app
`granted`	Concedido	Abre la cámara.
`prompt`	Nunca se preguntó	Muestra el diálogo del sistema (`requestPermissions`).
`prompt-with-rationale`	Lo negó una vez	Primero explica para qué se usa la cámara («Ahora no» / «Permitir») y, si acepta, muestra de nuevo el diálogo del sistema.
`denied`	**Negado de forma permanente**	Android ya no muestra el diálogo. La app muestra la alerta **«Cámara bloqueada»** con **Abrir ajustes** (`openSettings`) o **Escribir el código**, y deja un aviso fijo con el mismo botón.
(navegador o sin cámara)	No disponible	No muestra el botón de escanear; se escribe el código.

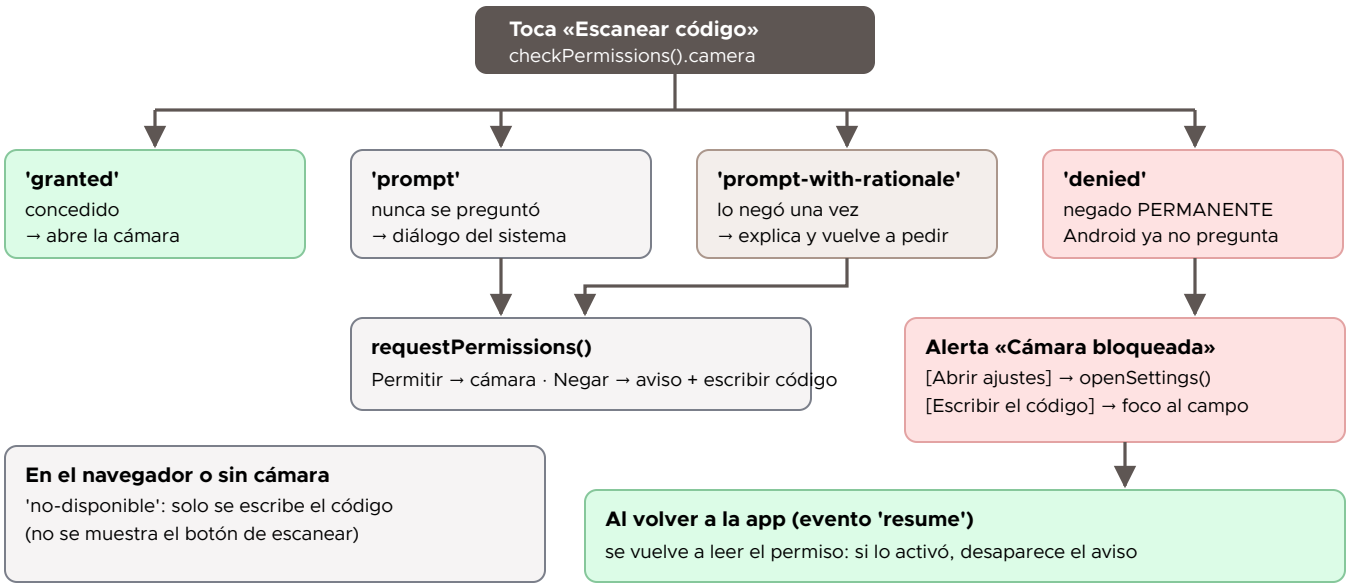


Figura 1. Flujo del permiso de cámara según su estado.

3.3 Denegación permanente

- **Cómo se detecta:** al responder el diálogo, Capacitor consulta a Android (`shouldShowRequestPermissionRationale`). Si ya no se puede volver a preguntar («No volver a preguntar», o negado dos veces desde Android 11), el estado queda en `denied`; si todavía se puede, en `prompt-with-rationale`.
- **Qué ve el usuario:** la alerta «Cámara bloqueada» explica que Android ya no pregunta y ofrece **Abrir ajustes** (abre la pantalla de la app en Ajustes → Permisos) o **Escribir el código** (lleva el foco al campo). En la pantalla queda además un aviso fijo con el botón Abrir ajustes.
- **Al volver de Ajustes:** con el evento `resume` la app vuelve a leer el permiso. Si el usuario lo activó, el aviso desaparece y ya puede escanear, sin reiniciar la app.
- **Nunca se bloquea el trabajo:** con el permiso negado (una vez o para siempre) la recepción se completa escribiendo el código.

3.4 Uso responsable de la cámara

- La cámara solo está encendida mientras se escanea. Se apaga al leer el primer código, al tocar Cancelar, con el botón atrás, al salir de la pantalla o si la app pasa a segundo plano (`pause`).
- Las imágenes se procesan en el teléfono (ML Kit con el modelo incluido): no se guardan ni se envían. Solo se envía a la API el texto del código.

3.5 Tráfico HTTP sin cifrar (solo desarrollo)

La API del laboratorio corre por HTTP (`php artisan serve`). Android bloquea ese tráfico por defecto, así que se permite **solo** hacia los equipos de desarrollo, en `android/app/src/main/res/xml/network_security_config.xml` (`10.0.2.2` para el emulador y `localhost` para el teléfono por USB con `adb reverse`), y con `android.allowMixedContent` en `capacitor.config.ts`. En producción, la API usaría HTTPS y se quitarían ambos.

3.6 Permisos según el rol del usuario

Además de los permisos del sistema, la API limita lo que puede hacer cada rol. El login devuelve las *abilities* del token y la app las usa en tres niveles:

- **Interfaz:** `sesion.puede('permiso')` oculta pestañas y botones que el rol no puede usar.
- **Rutas:** cada ruta privada exige un permiso (`meta.permiso`); si falta, el guardián redirige al inicio.
- **API:** si el backend responde 403, se muestra su mensaje.

Función en la app	Permiso (ability)	admin	cajero	bodeguero
Ver productos y su detalle	`view products`	Sí	Sí	Sí
Crear y editar productos	`create / update products`	Sí	No	Sí
Eliminar productos	`delete products`	Sí	No	No
Ver historial de ventas	`view sales`	Sí	Sí (las suyas)	No
Registrar ventas	`create sales`	Sí	Sí	No
Editar ventas	`update sales`	Sí	No	No
Recepción con la cámara	`receive purchases`	Sí	No	Sí